



**UNIVERSITATEA
TEHNICĂ
DIN CLUJ-NAPOCA**

UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA
CATEDRA DE CALCULATOARE

Sistem de control al semaforului inteligent

Student:

Stolniceanu Iustin-Pavel
Grupa 30237

Îndrumător proiect:

Șl. Dr. Ing. Darius Cătălin
Corpordean

9 ianuarie 2026

Cuprins

1	Rezumat	3
2	Introducere	5
2.1	Obiectivele proiectului	5
3	Fundamentare teoretică	5
3.1	Principii de control adaptiv – Algoritmul Queue Max-Pressure	6
3.2	Simularea traficului (SUMO + TraCI)	6
3.3	Comunicarea serială în sisteme Embedded	7
4	Proiectare și implementare	7
4.1	Metoda experimentală și soluția aleasă	7
4.2	Arhitectura generală a sistemului	7
4.3	Detalii de implementare	8
4.3.1	Implementarea comunicării UART custom (FPGA)	8
4.3.2	Implementarea software a sistemului de control trafic	8
4.3.3	Logica de simulare și generarea traficului (Python)	9
4.4	Alternative de proiectare analizate	10
4.5	Manual de utilizare	10
5	Rezultate experimentale	11
5.1	Mediul de lucru și instrumentele utilizate	11
5.2	Utilizarea Resurselor Hardware (FPGA)	12
5.3	Procedura de testare și Scenarii	12
5.3.1	Scenariul 1: Trafic Dezechilibrat	12
5.4	Analiză Comparativă: Static vs. Adaptiv	13
5.5	Dificultăți întâmpinate și soluții	13
6	Concluzii	14
6.1	Contribuții proprii	14

6.2	Directii viitoare de dezvoltare	15
-----	---	----

1 Rezumat

Congestia traficului urban reprezintă o provocare majoră a infrastructurii moderne, având un impact direct asupra calității vieții și a eficienței economice. Această lucrare propune proiectarea și implementarea unui sistem de control inteligent al semafoarelor, capabil să se adapteze dinamic la fluctuațiile fluxului de vehicule.

Arhitectura sistemului se bazează pe platforma de dezvoltare Zybo Z7000 (System-on-Chip), utilizând o abordare mixtă hardware-software. Logica de control implementează algoritmul *Queue Max-Pressure*, care optimizează fazele semaforului în funcție de lungimea cozilor de așteptare, procesată de procesorul ARM, în timp ce comunicarea achiziția de date este realizată de o simulare SUMO, iar un modul hardware dedicat asigură comunicarea prin UART cu ZYBO-ul.

Validarea soluției a fost realizată printr-o simulare de tip *Hardware-in-the-Loop* (HIL), integrând mediul de simulare SUMO (Simulation of Urban Mobility) cu placa de dezvoltare pentru a modela o intersecție reală din municipiul Botoșani, pe care am modelat-o manual în SUMO cu ajutorul utilitarului Netedit [5].



Figura 1: Intersecția străzilor Calea Națională și Sucevei din Botoșani - Vedere de ansamblu ((Sursă: Google Maps [6]))

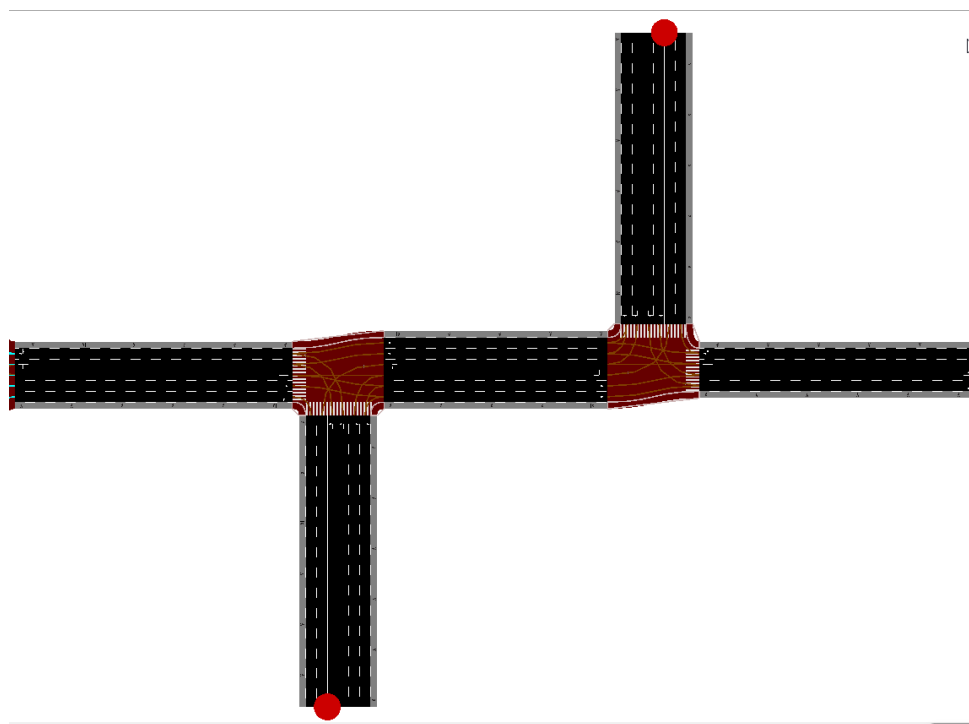


Figura 2: Intersectia din viata reala, modelata in Netedit

Rezultatele experimentale demonstrează eficiența sistemului propus, obținându-se o reducere a timpului mediu de așteptare cu aproximativ 16% comparativ cu un sistem de semaforizare static, validând astfel potențialul algoritmilor adaptivi în fluidizarea traficului urban.

2 Introducere

De prea multe ori se întâmplă să stai la semafor degeaba. Prin utilizarea unui sistem inteligent bazat pe un Zybo, putem încerca să eliminăm timpii morți la semafoare (timp mort = când nimeni care așteaptă la semafor nu are verde), astfel fluidizând traficul rutier și pietonal.

Sistemele de control al traficului reprezintă o componentă esențială în contextul dezvoltării continue a orașelor. Conform unui studiu publicat în revista Nature [1], aceste sisteme de control pot reduce timpul petrecut în trafic cu 11%, și, dacă sunt aplicate pe scară largă, pot reduce semnificativ noxele și emisiile de CO₂.

Întrucât acesta este doar un proiect demonstrativ, vom simula traficul folosind SUMO (Simulation of Urban Mobility). În viața reală, valorile de trafic pot proveni de la camere video, senzori de proximitate, senzori în asfalt, etc.

Problema principală abordată în această lucrare este optimizarea funcționării unui semafor astfel încât timpii de așteptare la semafor să fie dinamici și bazați pe datele primite de la senzori. Voi arăta cum poate fi optimizată o intersecție reală din Botoșani cu ajutorul unei plăcuțe Zybo. Scopul este reducerea timpilor morți.

2.1 Obiectivele proiectului

Obiectivele proiectului includ:

- Proiectarea și implementarea unui model funcțional de semaforizare inteligentă pentru o intersecție reală. Conceptele prezentate în lucrare pot fi extinse și pentru alte tipuri de intersecție.
- Utilizarea unor valori de trafic simulate (generate de SUMO), pentru detectarea traficului pietonal și auto.
- Dezvoltarea unei logici de control adaptiv, capabilă să ajusteze timpii de așteptare în funcție de datele primite.

Soluția propusă presupune proiectarea unui sistem de control bazat pe placa Zybo, care să se conecteze printr-un protocol UART custom la un calculator care rulează un server de SUMO. Avantajele unui astfel de sistem sunt evidente în comparație cu un semafor clasic: poate reacționa dinamic la condițiile reale de trafic, reducând timpii morți și sporind eficiența generală.

3 Fundamentare teoretică

Controlul adaptiv al semafoarelor este o aplicație practică a sistemelor încorporate de tip System-on-Chip, precum Zybo Zynq-7000, care integrează într-un singur chip atât un procesor ARM, cât și o zonă de logică programabilă FPGA. Un astfel de sistem poate

analiza date provenite de la senzori (sau de la o simulare) și poate ajusta automat durata semafoarelor în funcție de densitatea traficului, contribuind la fluidizarea circulației.

3.1 Principii de control adaptiv – Algoritmul Queue Max-Pressure

Voi încerca să prezint, pe scurt, algoritmul folosit de mine, Queue Max-Pressure [2]. Acest algoritm este folosit pentru rețele de intersecții, dar eu îl voi simplifica și îl voi folosi pentru a controla două intersecții. Acest algoritm este unul distribuit, adică rulează individual pentru fiecare intersecție, fără a lua în considerare ce fac celelalte intersecții.

Concepte fundamentale ale algoritmului:

- **Upstream Link (l):** Reprezintă segmentul de stradă care intră în intersecție (unde așteaptă mașinile).
- **Downstream Link (m):** Reprezintă segmentul de stradă care iese din intersecție.
- **Fază a semaforului:** Un set de manevre non-conflictuale care pot primi simultan lumina verde (exemplu: înainte Nord-Sud).
- **Queue length (Q):** Reprezintă numărul de mașini care așteaptă la semafor.
- **Pressure (P):** O măsură a dezechilibrului pentru o anumită fază. Reprezintă beneficiul obținut în fluxul de trafic prin acordarea culorii verzi unei anumite faze.

Formule matematice: Presiunea exercitată de o legătură l către o legătură m este definită ca diferența dintre coada de intrare și cea de ieșire:

$$P_{l \rightarrow m} = Q_l - Q_m \quad (1)$$

Interpretare: Cu cât este mai mare presiunea, cu atât este mai urgent să acordăm verde acelei faze. O presiune mare înseamnă că coada *downstream* este mică și că traficul are o cale clară de ieșire.

Presiunea totală a unei faze (P_{phase}) este suma presiunilor tuturor conexiunilor permise în acea fază:

$$P_{phase} = \sum_{l \in L_{active}} (Q_l - Q_{out}) \quad (2)$$

Regula de control: Politica fundamentală a algoritmului Max Pressure este de a selecta faza care are presiune maximă. Această regulă de decizie este demonstrată teoretic că maximizează debitul total al traficului [2].

3.2 Simularea traficului (SUMO + TraCI)

Pentru vizualizarea algoritmului de trafic fără a-l implementa în viața reală, se utilizează simulatorul SUMO (Simulation of Urban Mobility) [3]. SUMO este o platformă open-source utilizată pe scară largă pentru acest scop.

Pentru a permite controlul semafoarelor în timp real din exteriorul simulării, se utilizează protocolul **TraCI** (Traffic Control Interface). Acesta funcționează pe o arhitectură client-server (TCP), permițând unui program extern (în cazul acesta, un script de Python care comunică serial cu ZYBO-ul) să citească valorile de trafic și să modifice starea semafoarelor în orice moment, după cum dictează algoritmul de semaforizare.

3.3 Comunicarea serială în sisteme Embedded

Interfațarea între module hardware distincte (unitatea de calcul ZYBO și unitatea de afișare – în acest caz, un laptop) necesită protocoale de comunicare. Se folosește UART (Universal Asynchronous Receiver-Transmitter) [4]. Acesta este un protocol de comunicație serială asincronă care permite schimbul de date între două dispozitive. Acest protocol este implementat direct în FPGA.

4 Proiectare și implementare

4.1 Metoda experimentală și soluția aleasă

Pentru realizarea acestui proiect, am utilizat o metodă mixtă ce combină o implementare hardware, în FPGA, a protocolului UART pe Zybo Zynq-7000 cu o implementare software în C a logicii de control și o implementare software în Python (cu ajutorul librăriei TraCI) a logicii de simulare a traficului și a achiziției de date.

Soluția aleasă se bazează pe separarea logicii de control (executată pe ZYBO) de mediul de simulare (SUMO). Această abordare este preferată deoarece oferă o modularitate ridicată și permite testarea algoritmului în condiții de trafic realiste, imposibil de obținut printr-o simulare statică. Simularea poate fi înlocuită, fără mari bătaii de cap, cu un sistem de semaforizare real.

4.2 Arhitectura generală a sistemului

Sistemul este compus din două module principale care comunică între ele prin protocolul UART:

- **Modulul de simulare (PC):** Rulează Eclipse SUMO și un script Python (interfața TraCI) pentru vizualizarea simulării și pentru generarea fluxului de vehicule.
- **Modulul de procesare (Zybo 7000):** Acesta conține:
 - **Hardware (PL):** Un controller UART custom pentru recepția și transmisia datelor. Conține un buffer circular de 64 Bytes pentru stocarea datelor primite de la simulare. Mai conține și un IP XADC, pentru citirea datelor de la un potentiometru. Diagrama completa poate fi văzută în Anexa A.
 - **Software (PS):** Rulează algoritmul Queue Max-Pressure și gestionează, în timp real, timpii de semaforizare.

4.3 Detalii de implementare

4.3.1 Implementarea comunicării UART custom (FPGA)

În loc să utilizez IP-ul standard, am decis, în scop didactic, să îmi creez propriul modul UART, pornind de la implementarea din Laboratorul 1 [4]. Modulul a fost scris în limbajul VHDL și este proiectat să funcționeze la o frecvență de ceas de 100 MHz și comunică la un baud-rate de 9600.

Implementarea se bazează pe 3 submodule principale: generatoarele de semnal pentru sincronizare, automatele de stări finite pentru transmisie/recepție și un buffer circular pentru gestionarea fluxului de date asincron.

A. Sincronizarea și baud rate. Pentru a asigura comunicarea cu PC-ul, am implementat două divizoare de frecvență pe baza de numărătoare. Pentru transmisie, se împarte frecvența de 100 MHz la baud rate-ul dorit și se obține numărul necesar, la care trebuie împărțit clock-ul. Pentru recepție, se folosește o tehnică de supra-eșantionare (*oversampling*) de 16 ori mai rapidă decât rata de transfer. Codul poate fi găsit în Anexa B.

B. Buffer Circular pentru recepție. Se folosește de doi pointeri pentru a implementa un FIFO Queue. Această arhitectură permite stocarea temporară a comenzilor venite de la PC în timp ce procesorul Zybo execută alte sarcini. Astfel, PS-ul nu trebuie să verifice de 9600 de ori pe secundă dacă a primit ceva sau nu. Codul poate fi găsit în Anexa B.

C. Interfațarea între PL și PS. Deoarece logica UART rulează în partea de PL, iar algoritmul de trafic rulează pe procesorul ARM (PS), comunicarea între cele două domenii se realizează prin intermediul unor blocuri AXI GPIO. Schema detaliată a interconectării modulelor IP și a interfețelor AXI GPIO este prezentată în **Anexa A**.

4.3.2 Implementarea software a sistemului de control trafic

Logica de nivel înalt a sistemului rulează pe procesorul ARM integrat în ZYBO. Aplicația este scrisă în limbajul C și urmează o arhitectură Layered, menită să separe detaliile hardware de algoritmi de control ai traficului.

A. Hardware Abstraction Layer (HAL) Pentru a asigura portabilitatea codului și pentru a simplifica logica principală, am implementat un HAL. Acesta decuplează metodele și apelurile de sistem specifice Xilinx. Astfel, în controller-ul de trafic, în loc să citesc valoarea potențiometrului cu un apel specific (ex: `XAdcPs_GetAdcData`), doar apelez `read_potentiometer()` din HAL. Această abordare permite ca, în cazul schimbării platformei hardware, să fie necesară doar rescrierea fișierului `HAL.c`. Această abordare mi-a permis să pot testa codul acasă, fără a dispune de un Zybo, cu ajutorul unui Arduino UNO și a unui HAL specific. Codul poate fi găsit în Anexa D.

B. Modelarea intersecției Intersecția este modelată sub forma unui graf orientat. Această structură este dinamică și configurabilă, permițând definirea oricărei topologii de intersecții, în limita memoriei disponibile. Fiecare nod reprezintă o bandă de circulație, fiecare bandă având un ID, un `car_count` (numărul de mașini care se află pe acea bandă) și culoarea semaforului. Muchiile din graf reprezintă conexiuni între benzi de intrare (ups-

tream) și benzi de ieșire (downstream). O fază este reprezentată de un array de conexiuni non-conflictuale, care pot primi culoarea verde simultan. Pentru fiecare intersecție care se dorește a fi modelată, este nevoie de o codificare a fiecărei benzi a intersecției cu numere începând de la 0. Codul poate fi găsit în Anexa D.

C. Logica de control și Algoritmul Max-Pressure Nucleul decizional al sistemului este implementat în `TrafficController.c` și funcționează pe baza unui FSM (Finite State Machine). Controlerul nu doar ciclează stările, ci calculează dinamic presiunea fiecărei faze posibile:

- **Calculul Presiunii:** Pentru fiecare fază, algoritmul iterează prin toate conexiunile asociate și aplică formula $P = \sum(Q_l - Q_m)$. Codul accesează structurile de date `Lane` populate anterior și face diferența dintre numărul de mașini de pe banda de intrare și cea de ieșire.
- **Mecanismul Anti-Starvation:** Pentru a preveni situația în care o fază cu trafic redus nu primește niciodată verde, am implementat un prag de timp maxim. Dacă o fază nu a fost servită pentru o perioadă mai lungă decât acest prag, algoritmul îi acordă prioritate maximă.
- **Gestionarea Pietonilor:** Sistemul tratează cererile pietonilor ca pe o "fază critică". La apăsarea butonului, funcția `signal_pedestrian` injectează o valoare artificială mare în contorul benzii de pietoni, forțând algoritmul să comute semaforul către aceasta.
- **Codul poate fi găsit în Anexa D**

D. Protocolul de Comunicație și Sincronizare Sincronizarea între simulatorul SUMO și logica de pe FPGA se realizează printr-un protocol serial robust de tip "Request-Response", implementat în scriptul `run_simulation.py` (Anexa C). Scriptul Python acționează ca mecanMaster și controlează pașii simulării. Datele sunt serializate într-un pachet de octeți delimitat de `0xFF` pentru a preveni desincronizarea.

4.3.3 Logica de simulare și generarea traficului (Python)

Scriptul Python (`run_simulation.py`) îndeplinește rolul de orchestrator al întregului sistem. Acesta nu doar vizualizează starea simulării, ci controlează activ fluxul de vehicule și sincronizarea timpului. Logica este implementată într-o buclă infinită care avansează simularea pas cu pas (*simulation step*), condiționat de primirea datelor de la placa FPGA.

Un element distinctiv al implementării este capacitatea de interacțiune în timp real prin intermediul hardware-ului. Scriptul decodifică pachetul primit prin UART de la placa Zybo, care conține, pe lângă stările semafoarelor, și valoarea curentă a potențiometrului (citită prin XADC). Această valoare analogică este normalizată și utilizată ca o variabilă de probabilitate pentru generarea vehiculelor noi în rețea.

Astfel, utilizatorul poate modifica densitatea traficului din simularea software prin simpla rotire a potențiometrului fizic de pe placa de dezvoltare, testând astfel reacția algoritmului Max-Pressure la schimbări bruște de congestie (flux scăzut vs. oră de vârf).

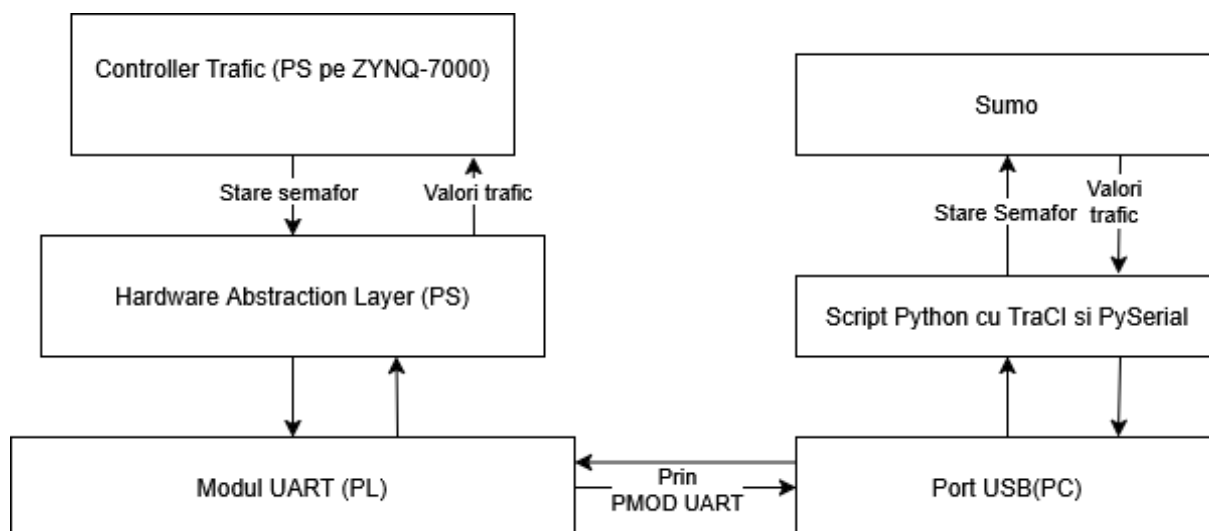


Figura 3: Schema bloc a arhitecturii hardware-software distribuite

4.4 Alternative de proiectare analizate

Pe parcursul dezvoltării prototipului, au fost analizate și alte soluții tehnice, care au fost ulterior abandonate în favoarea arhitecturii curente:

- **Simulare fizică cu potențiometre:** Inițial, s-a considerat utilizarea potențiometrelor conectate la pinii XADC ai plăcii Zybo pentru a simula densitatea traficului. Această metodă a fost abandonată deoarece nu permitea modelarea cozilor de așteptare dinamice și a interdependenței dintre intersecții, oferind un mediu de testare prea simplist pentru algoritmul Max-Pressure. S-a mers totuși pe ideea că potențiometrele pot spori traficul deja existent.
- **Comunicație Paralelă și afișarea cu ajutorul unui Arduino:** S-a analizat opțiunea de a transmite stările semafoarelor pe 8-16 fire paralele (GPIO) către sistemul de afișare, care ar fi trebuit să fie un Arduino UNO, care să se conecteze mai departe la niște shift-registeruri, care să se conecteze la niște LED-uri. Soluția a fost respinsă din cauza consumului ridicat de pini I/O și a complexității cablajului, preferându-se protocolul serial UART care necesită doar două fire (TX/RX) și simularea SUMO.

4.5 Manual de utilizare

Pentru replicarea experimentului și rularea sistemului, sunt necesare următoarele:

Cerințe preliminare:

- PC cu sistem de operare Windows/Linux și Python 3.8+ instalat.
- Simulatorul Eclipse SUMO (versiunea 1.18 sau mai nouă) și variabila de mediu SUMO_HOME configurată. Atenție! Instalarea SUMO va strica VITIS. Trebuie ștersă variabila de mediu PYTHON_PATH generată de SUMO.

- Placa Zybo Z7 conectată prin USB.
- PMOD Uart conectat la headerul JC și un cablu USB, cu un capăt conectat în calculator și un capăt conectat în PMOD Uart.

Pași de execuție:

1. Se deschide Vitis și se programează FPGA-ul cu ajutorul butonului RUN.
2. Pe PC, se navighează în folderul proiectului și se execută scriptul de simulare:

```
1 python run_simulation.py
2
```

3. Scriptul va deschide automat interfața grafică SUMO-GUI. Apăsați butonul *Play* în simulator pentru a începe sincronizarea cu placa.

5 Rezultate experimentale

În această secțiune sunt prezentate rezultatele obținute în urma implementării sistemului de control al traficului pe platforma Zybo Z7-20, precum și validarea acestuia prin simulare "Hardware-in-the-Loop" cu SUMO. Obiectivul principal al testelor a fost demonstrarea funcționalității protocolului de comunicație și eficiența algoritmului Max-Pressure comparativ cu un sistem static.

5.1 Mediul de lucru și instrumentele utilizate

Proiectarea și testarea sistemului au implicat utilizarea unui set divers de instrumente hardware și software, detaliate mai jos:

- **Platforma Hardware:** Digilent Zybo Z7-20 (SoC Xilinx Zynq-7000, Dual-Core ARM Cortex-A9 + FPGA Artix-7).
- **Sinteză Hardware (FPGA):** Vivado Design Suite 2025.1.
- **Dezvoltare Software (Embedded):** Vitis Unified Software Platform
- **Simulator Trafic:** Eclipse SUMO (Simulation of Urban Mobility) versiunea 1.18.0.
- **Interfațare și Automatizare:** Limbajul Python 3.9 cu biblioteca TraCI (Traffic Control Interface) și PySerial.
- **Sistem de Operare:** Windows 11 pe PC

5.2 Utilizarea Resurselor Hardware (FPGA)

Implementarea modulelor hardware personalizate (UART Controller, FIFO Buffers, AXI Interconnect) în logica programabilă (PL) a fost optimizată pentru a consuma un număr minim de resurse, lăsând spațiu disponibil pentru extinderi viitoare (ex: controlul mai multor intersecții simultan).

Sinteza proiectului în Vivado a generat următorul raport de utilizare a resurselor:

Tabela 1: Raport de utilizare a resurselor FPGA

Resursă	Utilizat	Total Disponibil	Grad de Utilizare (%)
Slice LUTs	1695	53,200	3.18%
Slice Registers (FF)	1946	106,400	1.83%
Block RAM (BRAM)	0	140	0.00%
IOW (Pini I/O)	12	200	6.00%
BUFG	1	32	3.12%

5.3 Procedura de testare și Scenarii

Validarea sistemului s-a realizat printr-o buclă de testare închisă (Closed-Loop Testing), urmând pașii:

1. **Initializare:** Scriptul Python generează topologia intersecției în SUMO și deschide portul serial.
2. **Generare Trafic:** Vehiculele sunt introduse în simulare cu o rată variabilă, controlată fie prin script, fie manual prin potențiometrul de pe placa Zybo.
3. **Feedback UART:** La fiecare pas de simulare (0.1s - 1s), SUMO trimite numărul de mașini către FPGA.
4. **Decizie:** FPGA-ul calculează noile faze și trimite comanda înapoi.
5. **Observație:** Se monitorizează vizual cozile de așteptare din SUMO.

5.3.1 Scenariul 1: Trafic Dezechilibrat

S-a simulat un scenariu în care fluxul de vehicule care vin din SUD este mult mai mare decât fluxul de vehicule care vin din alte direcții.

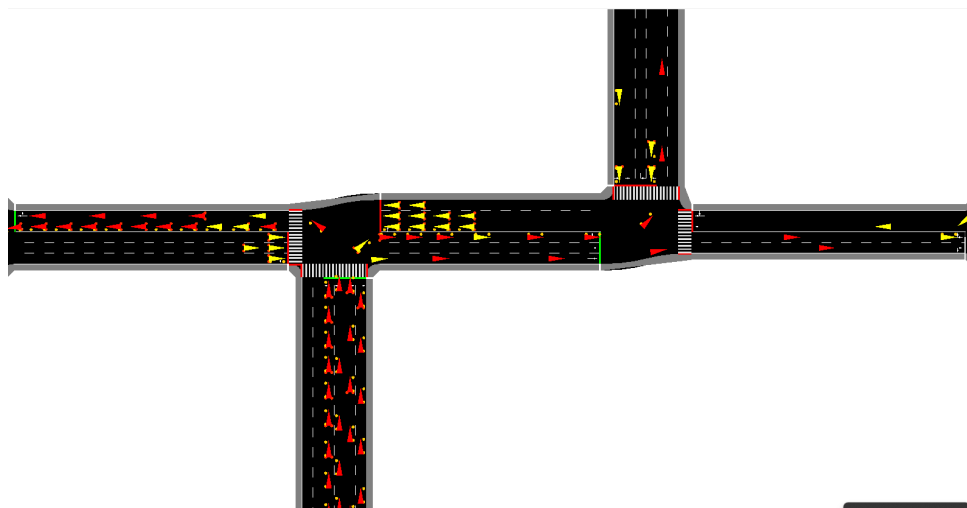


Figura 4: Captură de ecran din SUMO: Trafic intens din SUD

Sistemul Max-Pressure a reacționat prin alocarea unui timp de verde prelungit pentru axa aglomerată, reducând timpii de așteptare, în timp ce benzile mai libere au fost comutate pe roșu mai rapid.

5.4 Analiză Comparativă: Static vs. Adaptiv

Pentru a demonstra eficiența soluției, am comparat performanța sistemului implementat pe Zybo (Adaptiv) cu un controler clasic cu timpi fiși, rulând aceeași simulare timp de 60 minute (3600 de pași).

Tabela 2: Comparație performanță (Valori simulate)

Metrică	Sistem Static (Fix)	Sistem Adaptiv (Zybo)	Îmbunătățire
Timp mediu așteptare	26 secunde	21.85 secunde	$\approx 16\%$
Viteza Medie	5.86 m/s	6.29 m/s	7%
Timp mediu pierdut*	42.39s	38.33s	-4s

*Nota: Timpul mediu pierdut reprezintă intarzierea totala fata de o circulatie ideala. Acesta include atat timpul de stationare (waiting time), cat si timpul pierdut prin decelerare la franare si accelerare la plecarea de pe loc. Toate aceste metrici au fost calculate de SUMO.

5.5 Dificultăți întâmpinate și soluții

Pe parcursul implementării au apărut o serie de provocări tehnice:

1. **Sincronizarea UART:** Inițial, la viteze mari de transmisie, apăreau erori de încadrare ("framing errors") sau pachete trunchiate, ceea ce ducea la blocarea scriptului Python. **Soluție:** Am implementat delimitatori de pachet (0xFF) la începutul și sfârșitul fiecărei transmisii și am introdus un mecanism de *flush* al buffer-ului la

fiecare ciclu de simulare. De asemenea, se poate selecta în script-ul de Python un chunk-size, care reprezintă câți bytes vor fi trimisi de script în același timp către Zybo. Întrucât am observat că după al 7-lea byte încep să fie probleme la recepționarea datelor, am ales un chunk size mai mic - de 3, ca să fiu sigur că totul merge bine.

2. **Latenta comunicației:** Transmisia serială introduce o latență care poate desincroniza simularea SUMO (care rulează mult mai repede decât timpul real). **Soluție:** Am utilizat un protocol de tip Blocant (Handshake). SUMO așteaptă explicit răspunsul de la Zybo înainte de a calcula următorul pas de simulare.

6 Concluzii

Proiectul de față a reușit proiectarea și implementarea unui sistem complet de control al traficului de tip "Hardware-in-the-Loop", demonstrând viabilitatea utilizării tehnologiilor FPGA și a algoritmilor adaptivi în infrastructura urbană critică.

Obiectivul principal, acela de a reduce timpii morți din intersecții, a fost îndeplinit cu succes. Prin utilizarea algoritmului *Queue Max-Pressure*, rulat pe procesorul ARM al plăcii Zybo Z7-20, s-a obținut o reducere a timpului mediu de așteptare cu aproximativ 16% și o creștere a vitezei medii de deplasare cu 7%, comparativ cu un sistem convențional cu timpi ficși. Aceste rezultate, validate prin simularea microscopică SUMO a unei intersecții reale din Botoșani, confirmă faptul că o abordare dinamică este superioară uneia statică, în special în scenarii de trafic asimetric.

O provocare majoră a reprezentat-o sincronizarea între timpul simulat (software) și timpul de execuție al hardware-ului. Soluția adoptată, bazată pe un protocol de comunicație serială robust (Handshake) și module hardware personalizate (UART cu buffer circular), a asigurat o funcționare stabilă și fără pierderi de date.

6.1 Contribuții proprii

Principalele contribuții aduse prin această lucrare sunt:

- **Implementarea Hardware:** Dezvoltarea unui controller UART custom în limbajul VHDL, optimizat pentru resurse minime, în locul utilizării unor blocuri IP standard.
- **Implementarea Software:** Portarea algoritmului Max-Pressure pe arhitectura Zynq-7000 și crearea unui strat de abstractizare (HAL) pentru portabilitate.
- **Integrarea HIL:** Realizarea scriptului de automatizare în Python care sincronizează simulatorul SUMO cu placa de dezvoltare, permițând interacțiunea în timp real și controlul traficului prin potențiometru.
- **Modelarea Realistă:** Replicarea fidelă a geometriei intersecției Calea Națională - Sucevei din Botoșani în mediul de simulare folosind utilitarul Netedit.

6.2 Direcții viitoare de dezvoltare

Pentru a transforma acest prototip într-un produs comercial viabil sau pentru a extinde cercetarea, se propun următoarele direcții:

- **Extinderea la nivel de rețea:** Implementarea algoritmului pentru coordonarea mai multor intersecții adiacente, facilitând crearea unor unde verzi dinamice.
- **Upgrade de comunicație:** Înlocuirea protocolului UART cu Ethernet sau Wi-Fi (folosind un modul ESP32) pentru a permite monitorizarea de la distanță și viteze de transfer superioare.
- **Integrarea cu senzori reali:** Înlocuirea datelor simulate din SUMO cu date provenite de la camere video de procesare a imaginii (Computer Vision) sau senzori inductivi din asfalt.
- **Machine Learning:** Explorarea algoritmilor de *Reinforcement Learning* pentru ca sistemul să "învețe" tiparele de trafic specifice anumitor ore din zi și să anticipeze congestiile.

Bibliografie

- [1] Wu, K., Ding, J., Lin, J., et al., *Big-data empowered traffic signal control could reduce urban carbon emission*, Nature Communications, 2025. Disponibil la: <https://www.nature.com/articles/s41467-025-56701-4>
- [2] X. Li, G. Fan, W. Zhang, P. Wang, *Max-Pressure Controller for Traffic Networks Considering the Phase Switching Loss*, Sustainability, vol. 17, no. 10, p. 4492, May 2025. <https://www.mdpi.com/2071-1050/17/10/4492>
- [3] Eclipse Foundation, *SUMO Documentation - Simulation of Urban MObility*, 2025. <https://eclipse.dev/sumo/>
- [4] Mircea-Paul Mureșan, *Laboratorul 1 - Comunicarea Serială UART Implementare cu Automate cu Stări Finite*, Material didactic, Universitatea Tehnică din Cluj-Napoca, 2025.
- [5] DLR Institute of Transportation Systems, *Netedit Documentation - Graphical Network Editor*, 2024. Disponibil la: <https://sumo.dlr.de/docs/Netedit/index.html>
- [6] Google, *Google Maps – Vizualizare satelit a intersecției analizate, Botoșani*, 2026. Disponibil la: <https://maps.app.goo.gl/PBmaDptXRUj1vK5H7>